Proxmox, pfSense, Ubuntu DNS, WireGuard, and VLAN Wireless Network Deployment

Project Summary

Item	Details
Project focus	Secure home and IoT network segmentation using virtualization, firewalling, VPN access, internal DNS, and VLAN-based wired and wireless isolation
Core platforms	Proxmox VE, pfSense, Ubuntu Server, TP-Link managed switch, TP-Link access point
Security controls	Stateful firewalling, WireGuard remote access, VLAN segmentation, controlled port forwarding, internal DNS, and interface-level policy enforcement
Publication note	Sensitive details such as credentials, device secrets, and public-facing identifiers are intentionally omitted

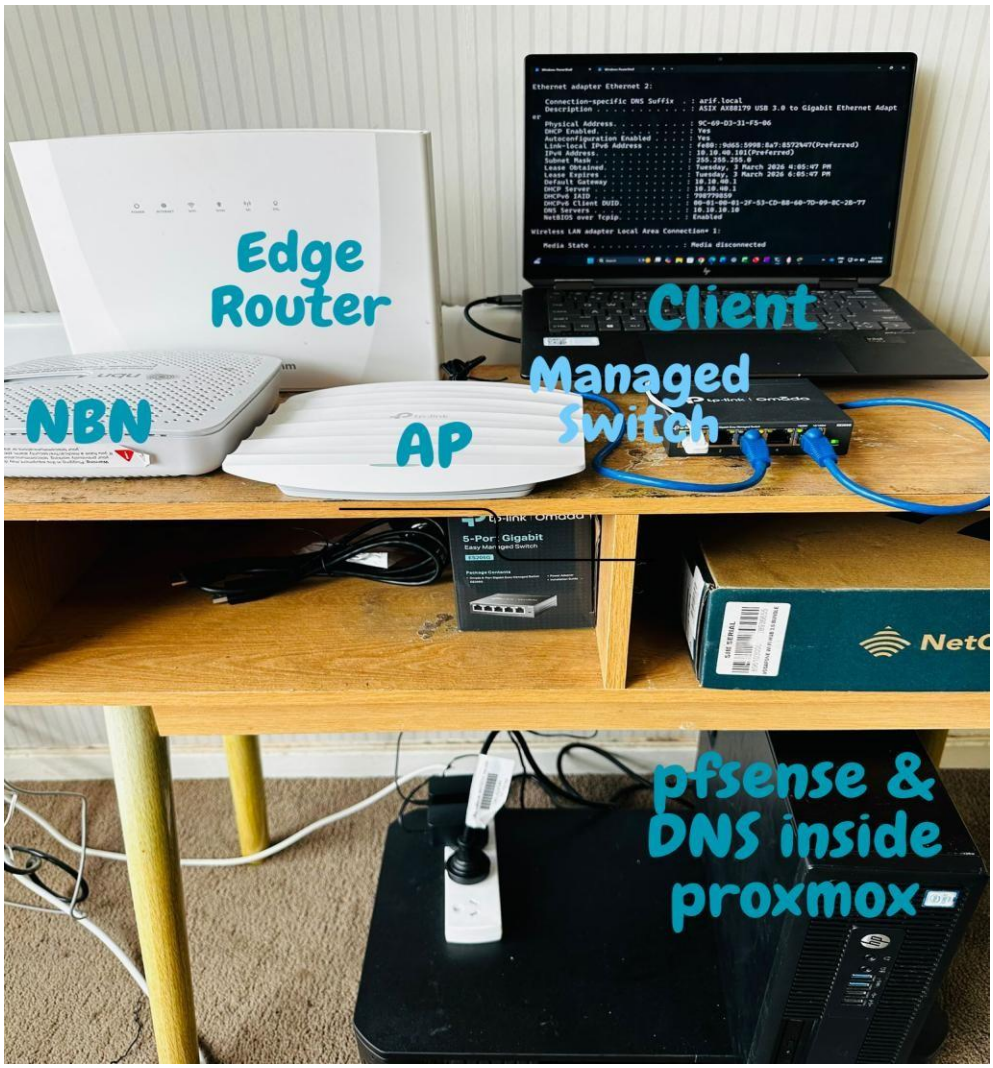


Figure 1: *Physical lab setup showing the NBN modem, EdgeRouter, TP-Link access point, managed switch, client laptop, and the Proxmox host running pfSense and the internal DNS server.*

1. Executive Summary

This project documents the design and implementation of a secure internal network built behind an existing EdgeRouter using Proxmox VE, pfSense, Ubuntu DNS, WireGuard, a managed switch, and a VLAN-aware wireless access point. The goal was not to replace the upstream router outright, but to introduce a controlled and isolated security layer behind it. This approach preserved internet availability on the existing home network while allowing the new environment to operate as a separate, firewall-controlled LAN.

pfSense was deployed as the main virtual firewall and router for the internal environment, providing stateful firewalling, DHCP, VLAN routing, NAT, controlled port forwarding, and WireGuard VPN access. Ubuntu Server was deployed behind pfSense as DNS1 to provide internal name resolution and utility services. The network was then extended through an additional NIC to a managed switch and access point, allowing multiple VLANs to be carried across both wired and wireless segments.

The final design demonstrates a secure home and IoT implementation with segmented networks for native management, NAS, IoT, general users, and guest access. Remote administration is delivered through WireGuard rather than broad internet exposure, and traffic between networks is controlled through pfSense interface-based firewall rules.

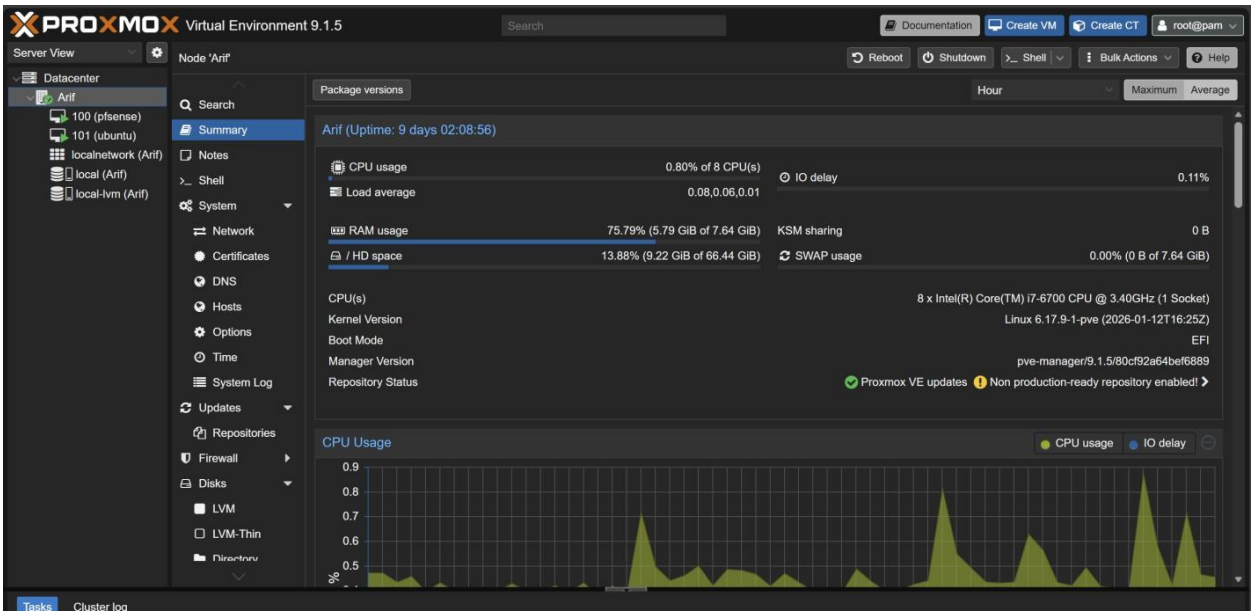


Figure 2: *Proxmox VE web dashboard for the host node, showing system uptime, CPU and RAM utilisation, and storage status for the virtualisation platform.*

2. Target Architecture and Security Model

The architecture separates upstream connectivity from the protected internal environment. The existing EdgeRouter continues to provide stable internet access to the home network, while pfSense, sitting behind it, creates a dedicated security boundary for the virtual lab and segmented internal devices. This design reduces disruption, preserves availability, and allows the internal environment to be managed independently.

2.1 Physical and virtual layout

- Internet / ISP router -> EdgeRouter -> desktop PC running Proxmox VE
- Proxmox vmbr0 -> upstream bridge used for host management and pfSense WAN
- Proxmox vmbr1 -> internal bridge / trunk side used for pfSense LAN, Ubuntu DNS1, switch, and AP
- pfSense WAN -> upstream private subnet behind the EdgeRouter
- pfSense LAN native network -> 10.10.10.0/24
- Managed switch -> carries native VLAN 1 and tagged VLANs 20, 30, 40, and 50
- Wireless AP -> managed on VLAN 1, additional SSIDs mapped to tagged VLANs
- WireGuard -> secure remote entry point into the protected internal environment

2.2 IP and VLAN plan

Segment / Purpose	VLAN	Subnet / Gateway	Notes
Native LAN / management	1	10.10.10.0/24, gateway 10.10.10.1	Used for Ubuntu DNS1, switch management, AP management, and general internal administration
NAS network	20	10.10.20.0/24, gateway 10.10.20.1	Tagged on trunks, wired access on a dedicated port, wireless SSID mapped to VLAN 20
IoT network	30	10.10.30.0/24, gateway 10.10.30.1	Tagged on trunks, dedicated access port and SSID for isolated IoT devices
General user network	40	10.10.40.0/24, gateway 10.10.40.1	User segment for non-IoT wireless and future wired clients
Guest network	50	10.10.50.0/24, gateway 10.10.50.1	Separate guest segment controlled through pfSense policy
WireGuard tunnel	-	10.6.0.0/24, pfSense 10.6.0.1	Remote VPN path used for secure administration and internal access

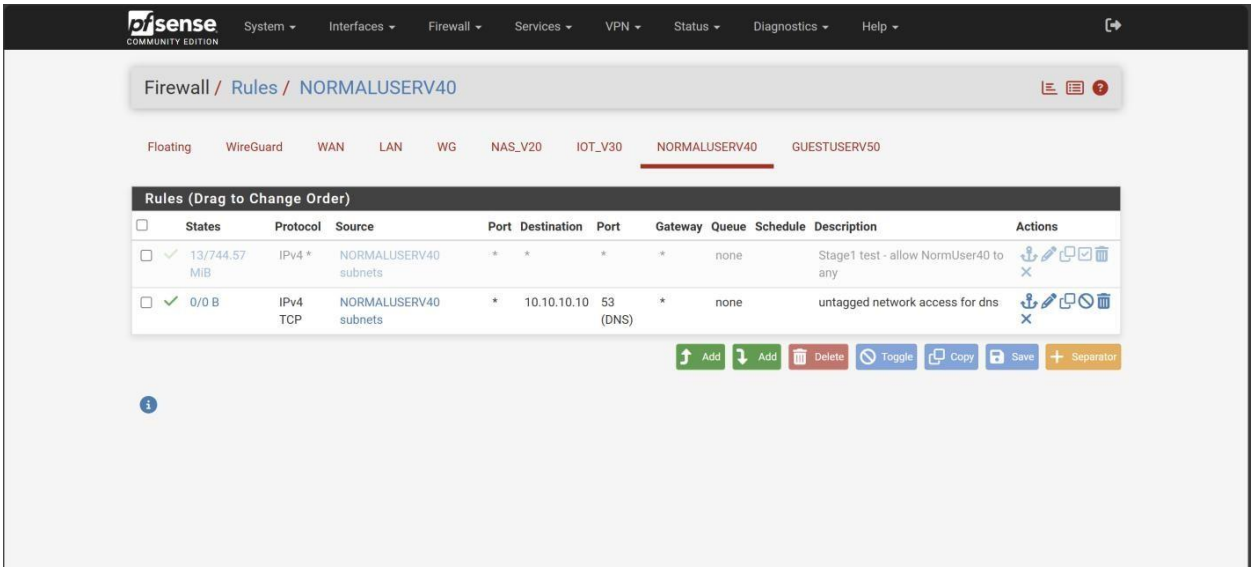


Figure 3: *pfSense firewall rules applied to the NORMALUSERV40 (VLAN 40) interface, showing a validation allow rule and an explicit rule permitting DNS traffic to the internal resolver.*

3. Proxmox Host Deployment

The build began by converting a standard desktop PC into a Proxmox VE host. After installing Proxmox from boot media, the host was brought online and accessed through the web interface. Two virtual switching domains were then created: one for upstream connectivity and one for the isolated internal environment.

3.1 Base installation

- Booted the desktop PC from Proxmox VE installation media and completed the host installation.
- Verified that the Proxmox web interface was reachable from the existing home network.
- Confirmed the node was stable before deploying pfSense and Ubuntu VMs.

3.2 Virtual bridges and extra NIC integration

- vmbr0 was bound to the primary physical NIC and used for upstream connectivity and Proxmox host management.
- vmbr1 was created as the internal bridge used by pfSense LAN, Ubuntu DNS1, and later the switch and AP trunk side.
- An additional physical NIC was attached to vmbr1 so the internal network could be extended physically to the managed switch.
- VLAN-aware mode was enabled on vmbr1 so tagged VLAN traffic could pass between pfSense, the switch, and the AP.

A key troubleshooting discovery was that Proxmox also operates its own firewall at the virtual NIC layer. The pfSense LAN NIC originally had the Proxmox firewall enabled, which blocked tagged VLAN 20 DHCP traffic before it could reach pfSense. Once that NIC-level Proxmox firewall option was disabled, DHCP and VLAN traffic passed correctly.

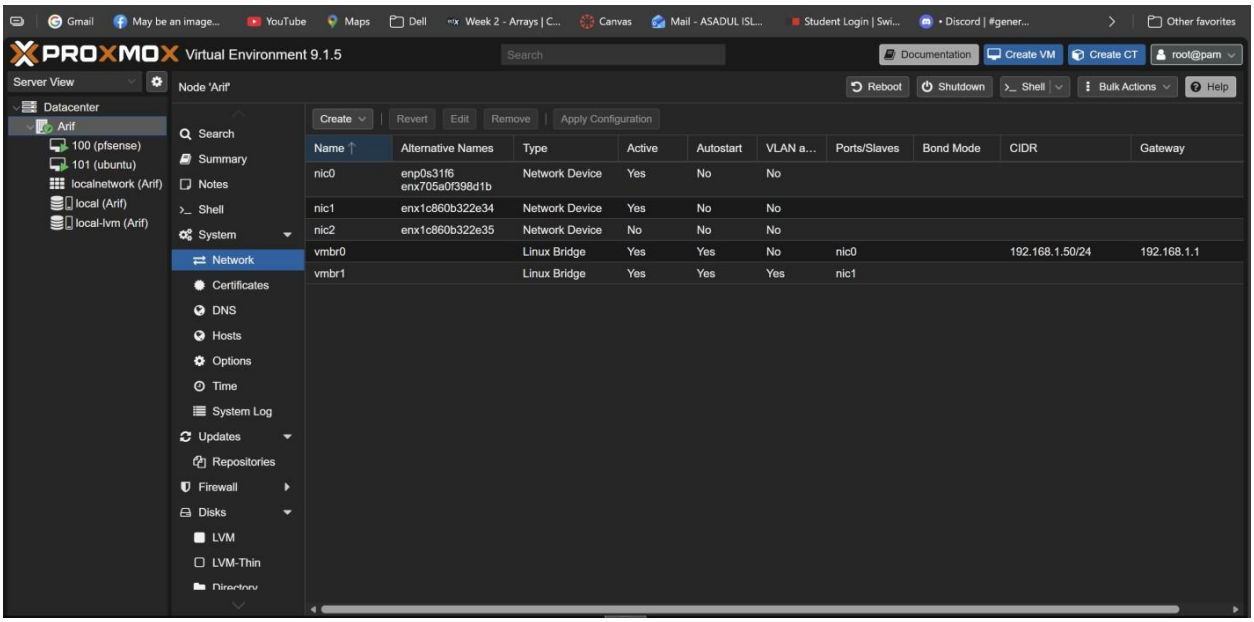


Figure 4: *Proxmox network configuration showing the physical NICs and the two Linux bridges (vmbr0 for upstream, vmbr1 for the internal VLAN trunk), with VLAN-aware mode enabled on vmbr1.*

4. pfSense Firewall and Core Network Services

pfSense was deployed as the main virtual firewall, router, and security control point for the isolated LAN. It handles WAN/LAN routing, DHCP, VLAN interfaces, NAT, firewall enforcement, and WireGuard, making it the primary policy engine for the environment. The EdgeRouter remains upstream as the existing internet gateway.

4.1 Installation and interface assignment

- Created a pfSense VM in Proxmox with one NIC on vmbr0 for WAN and one NIC on vmbr1 for LAN / VLAN trunking.
- Installed pfSense and verified the interface mapping through the pfSense console.
- Configured WAN to obtain upstream connectivity from the existing home network.
- Configured the native internal LAN as 10.10.10.1/24.

4.2 Why the EdgeRouter was retained upstream

The EdgeRouter was intentionally retained in the design. Keeping it upstream preserved a stable, already-working internet edge while pfSense introduced a separate protected LAN behind it. This allowed the new environment to be developed and secured without interrupting the existing household network. In practice, the EdgeRouter provides availability at the outer edge, while pfSense provides segmentation, firewall control, VPN access, and internal security inside the protected environment.

4.3 Administrative access and controlled exposure

During early deployment, WAN-side access to pfSense was temporarily used for setup because the workstation was initially located on the upstream side. Administrative services were not left broadly open; instead, HTTPS and SSH were enabled in a controlled manner and restricted where required. Once WireGuard became operational, VPN-based access became the preferred management path.

4.4 WAN NAT and service forwarding

Purpose	External listener	Internal target	Security note
WireGuard VPN	UDP 51820 on pfSense WAN	pfSense WireGuard service	Primary secure remote entry point
Ubuntu SSH	TCP 2222 on pfSense WAN	Ubuntu DNS1 port 22	Restricted for administration only
Internal DNS UI / service UI	TCP 5380 on pfSense WAN	Ubuntu DNS1 port 5380	Restricted and used only when needed

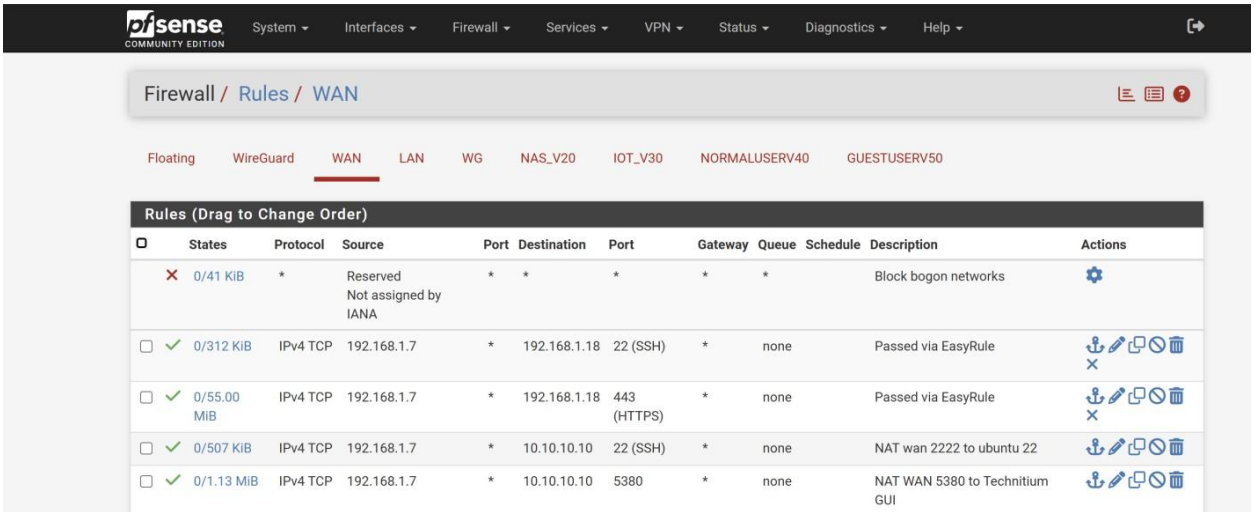


Figure 5: *pfSense WAN firewall rules, showing the default bogon block rule along with controlled NAT pass rules for administrative HTTPS, SSH, the Ubuntu DNS GUI, and the WireGuard listener.*

5. Ubuntu DNS1 Server

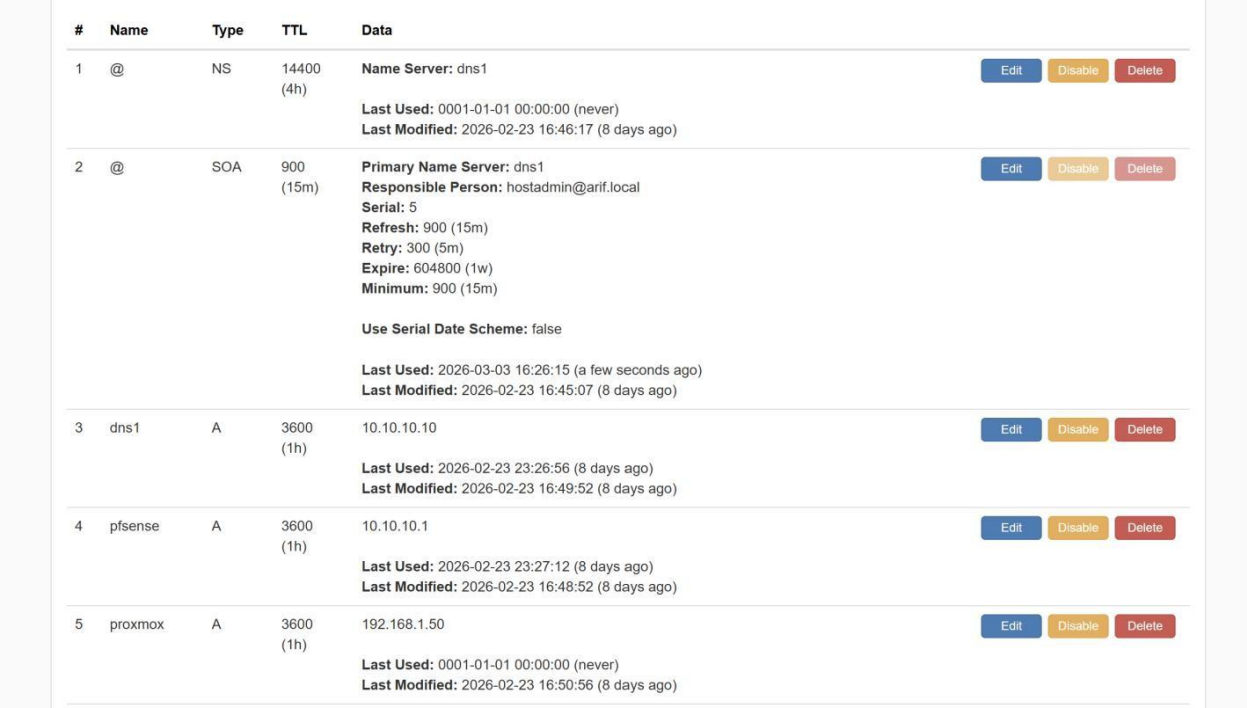
An Ubuntu Server VM was deployed behind pfSense on the internal network and used as DNS1 and a utility server. This host became an important internal service because it provided stable local name resolution for key systems such as pfSense, Proxmox, and other internal devices.

5.1 VM deployment and addressing

- Created the Ubuntu Server VM in Proxmox and attached it to vmbr1 behind pfSense.
- Initially tested with DHCP addressing, then moved to a stable internal address of 10.10.10.10.
- Used DHCP reservation / planned addressing so the DNS server remained consistent for clients and firewall references.

5.2 DNS configuration

- Configured the Ubuntu host as internal DNS1 for the environment.
- Created internal DNS records for important systems such as dns1, pfSense, and the Proxmox host.
- Referenced DNS1 in pfSense DHCP settings so clients could learn internal name resolution automatically.



#	Name	Type	TTL	Data	
1	@	NS	14400 (4h)	**Name Server:** dns1 **Last Used:** 0001-01-01 00:00:00 (never) **Last Modified:** 2026-02-23 16:46:17 (8 days ago)	Edit Disable Delete
2	@	SOA	900 (15m)	**Primary Name Server:** dns1 **Responsible Person:** hostadmin@arif.local **Serial:** 5 **Refresh:** 900 (15m) **Retry:** 300 (5m) **Expire:** 604800 (1w) **Minimum:** 900 (15m) **Use Serial Date Scheme:** false **Last Used:** 2026-03-03 16:26:15 (a few seconds ago) **Last Modified:** 2026-02-23 16:45:07 (8 days ago)	Edit Disable Delete
3	dns1	A	3600 (1h)	10.10.10.10 **Last Used:** 2026-02-23 23:26:56 (8 days ago) **Last Modified:** 2026-02-23 16:49:52 (8 days ago)	Edit Disable Delete
4	pfsense	A	3600 (1h)	10.10.10.1 **Last Used:** 2026-02-23 23:27:12 (8 days ago) **Last Modified:** 2026-02-23 16:48:52 (8 days ago)	Edit Disable Delete
5	proxmox	A	3600 (1h)	192.168.1.50 **Last Used:** 0001-01-01 00:00:00 (never) **Last Modified:** 2026-02-23 16:50:56 (8 days ago)	Edit Disable Delete

Figure 6: *Internal DNS zone on the Ubuntu DNS1 server, showing the NS and SOA records along with A records for dns1, pfsense, and proxmox used by internal clients for name resolution.*

6. WireGuard Secure Remote Access

WireGuard was deployed to provide secure remote access from outside the home network. Rather than exposing administrative services directly to the internet, the design uses a single VPN entry point and then allows management to occur from inside the protected environment.

6.1 Upstream router forwarding

- Configured the upstream router to forward the WireGuard UDP listening port to the pfSense WAN address.
- Verified that remote VPN traffic reached pfSense rather than internal hosts directly.

6.2 pfSense WireGuard configuration

- Configured a WireGuard instance on pfSense with a dedicated tunnel subnet.
- Assigned the WireGuard interface and applied firewall rules for the required remote access paths.
- Validated the tunnel through ping, route checks, curl testing, and SSH access to internal resources.

This provides the environment with a secure external management method that is well suited to a home and IoT deployment: remote administration is possible, but only after authenticated VPN access.

7. VLAN Interfaces, DHCP, and Firewall Policy

After the native internal network was stable, additional VLANs were created in pfSense to segment storage, IoT, user, and guest traffic. Each VLAN was created on the pfSense LAN parent interface and then assigned its own interface, gateway, DHCP scope, and firewall policy.

7.1 Interface creation and addressing

- Created VLAN 20, 30, 40, and 50 on the pfSense LAN parent interface.
- Assigned each VLAN as its own pfSense interface.
- Configured gateway addresses 10.10.20.1, 10.10.30.1, 10.10.40.1, and 10.10.50.1 respectively.

7.2 DHCP scopes

Interface	Subnet	Example pool	Notes
NAS_V20	10.10.20.0/24	10.10.20.100-199	Used for NAS segment and VLAN 20 wireless clients
IOT_V30	10.10.30.0/24	10.10.30.100-199	Used for IoT devices and isolated clients
GeneralUser_V40	10.10.40.0/24	10.10.40.100-199	General user wireless / future wired clients
GuestUser_V50	10.10.50.0/24	10.10.50.100-199	Guest segment controlled by dedicated policy

7.3 Firewall rules

Firewall rules were applied on a per-interface basis. For validation, simple allow rules were used first to prove the traffic path end to end, with DHCP and DNS explicitly allowed where needed. Once connectivity was confirmed, the design could be tightened further according to the required level of isolation. This interface-based model is central to why pfSense was chosen: each VLAN can be treated as its own security zone.

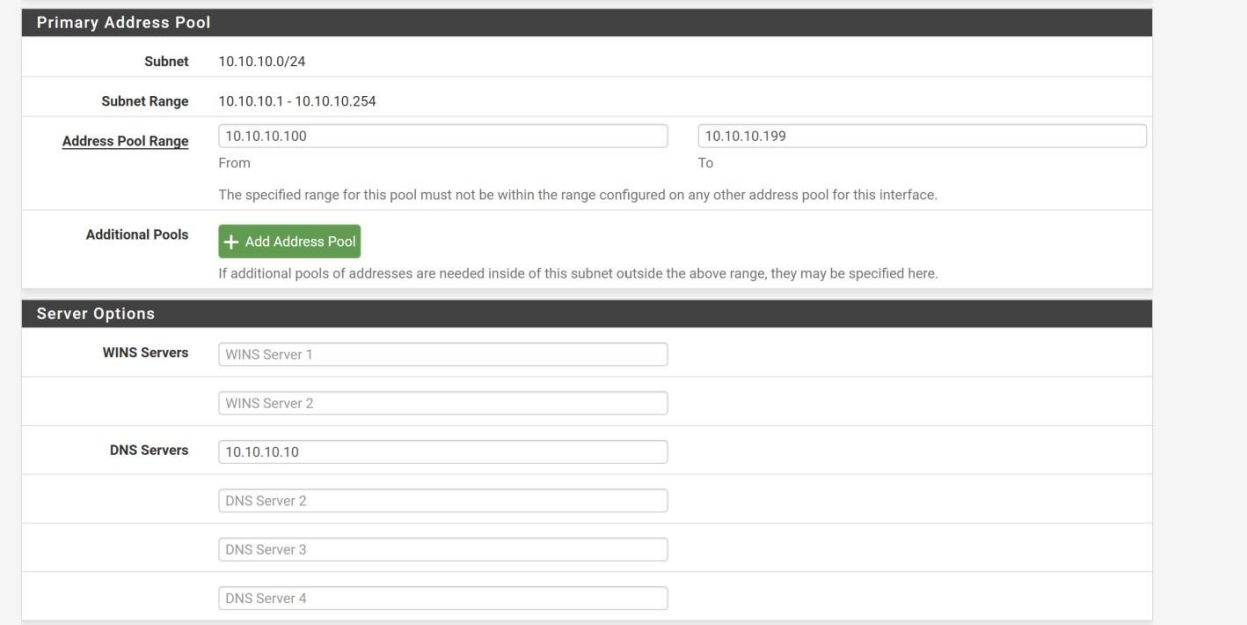


Figure 7: *pfSense DHCP server configuration for the native LAN, showing the primary address pool range and the internal DNS server (10.10.10.10) pushed to clients.*

8. Managed Switch Configuration

A TP-Link managed switch was connected to the internal Proxmox NIC and used to extend the segmented network physically. The switch carries native VLAN 1 plus tagged VLANs to the AP and provides wired access ports for selected segments.

8.1 Port design

Port	Role	VLAN membership	PVID
Port 1	Uplink to pfSense / Proxmox	VLAN 1 untagged; VLANs 20, 30, 40, 50 tagged	1
Port 2	Wired access for VLAN 20	VLAN 20 untagged; non-member of others after clean-up	20
Port 3	AP uplink	VLAN 1 untagged; VLANs 20, 30, 40, 50 tagged	1
Port 4	Wired access for VLAN 30	VLAN 30 untagged	30
Port 5	Native LAN / spare or future reassignment	VLAN 1 untagged unless reassigned	1

8.2 Switch logic used

- Tagged ports were used for trunk paths that needed to carry multiple VLANs.
- Untagged ports were used for ordinary end devices that do not understand VLAN tags.

- PVID defined which VLAN untagged ingress traffic belongs to on each access or hybrid port.
- Displayed VLAN names on the switch were only labels. Functional matching depends on VLAN ID, not text labels.

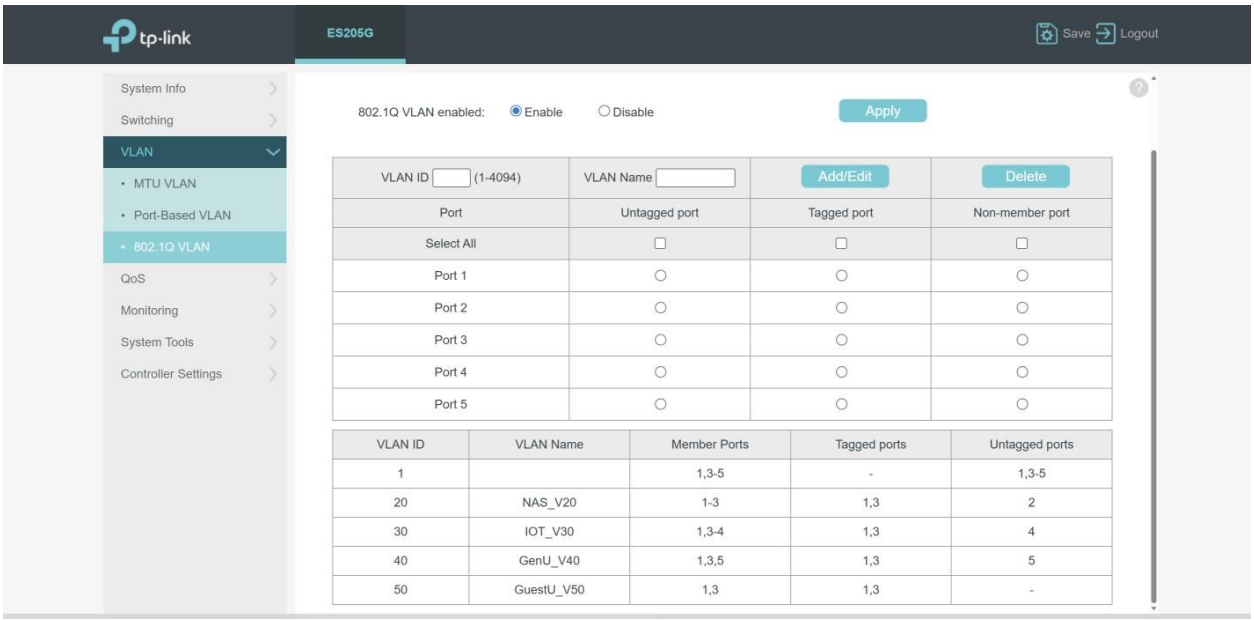


Figure 8: *TP-Link managed switch 802.1Q VLAN configuration, showing VLAN IDs 1, 20, 30, 40, and 50 with their corresponding tagged trunk ports and untagged access ports.*

9. Access Point Configuration and Wireless VLAN Mapping

The TP-Link AP was connected to the switch on a trunk-style uplink so that it could carry native management traffic plus tagged SSIDs. The AP itself remained on the native 10.10.10.0/24 management network, while individual SSIDs were mapped to additional VLANs for wireless segmentation.

9.1 AP management and SSID mapping

SSID purpose	AP VLAN setting	Expected subnet
Default / native SSID	VLAN disabled	10.10.10.0/24
NAS_V20	VLAN 20	10.10.20.0/24
IOT_V30	VLAN 30	10.10.30.0/24
General user SSID	VLAN 40	10.10.40.0/24
Guest SSID	VLAN 50	10.10.50.0/24

A key validation point was confirming that the AP management IP and the SSID VLAN IDs are separate concepts. The AP can remain managed on 10.10.10.x while still serving clients into VLAN 20, 30, 40, or 50. If an SSID does not receive the expected subnet, the most common cause is that VLAN tagging on that SSID is disabled or not applied.

Add

ID	SSID	VLAN ID	SSID Broadcast	Security Mode	Guest Network	Action
1	Def_AP_AR	0	Enable	WPA-Personal	Disable	
2	NAS	20	Enable	WPA-Personal	Disable	
3	IoT	30	Enable	WPA-Personal	Disable	
4	GenU	40	Enable	WPA-Personal	Disable	
5	GuestU	50	Enable	WPA-Personal	Disable	

Figure 9: *TP-Link access point SSID configuration, mapping each SSID to its corresponding VLAN ID (0 for the native management SSID, and 20, 30, 40, 50 for the segmented SSIDs).*

10. Troubleshooting Summary

The final deployment was validated through structured troubleshooting across Proxmox, pfSense, the switch, the AP, and client devices. The most significant issue was a VLAN 20 DHCP failure that initially appeared to be a switch or AP problem. Packet capture on pfSense showed no DHCP requests arriving on NAS_V20, which confirmed that the fault was upstream of pfSense. The actual root cause was the Proxmox firewall option enabled on the pfSense LAN NIC. Once that was disabled, VLAN 20 traffic reached pfSense and clients received the correct leases.

- Early WAN administration behaviour was validated by checking service listeners, controlled rules, and access from approved sources.
- Testing separated direct upstream access from WireGuard VPN access so interface-specific behaviour could be understood correctly.
- Browser behaviour and command-line behaviour over WireGuard were compared using curl, SSH verbosity, and routing checks.
- VLAN 20 APIPA addressing was traced through packet capture and ultimately linked to Proxmox NIC firewall filtering.
- Wireless SSIDs that received native 10.10.10.x addresses were corrected by enabling the proper VLAN ID on the AP SSID.
- Switch membership, tagged/untagged logic, and PVID assignments were validated to ensure trunks and access ports behaved as intended.

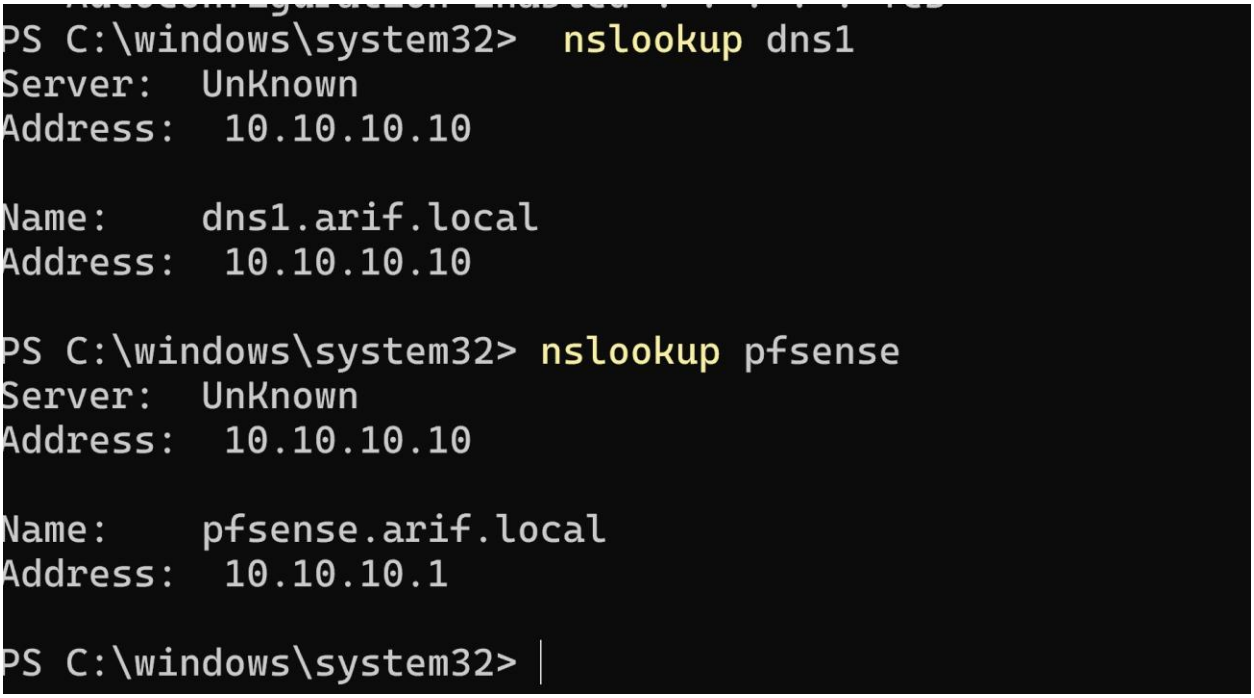


Figure 10: *Windows client nslookup output verifying internal DNS resolution: dns1.arif.local resolves to 10.10.10.10 and pfsense.arif.local resolves to 10.10.10.1, both via the internal DNS server.*

11. Outcome

The completed environment demonstrates practical infrastructure and security skills across virtualization, firewalling, VPN deployment, DNS, NAT, VLAN segmentation, switch configuration, wireless VLAN mapping, and troubleshooting. More importantly, it demonstrates a security-first design approach: instead of exposing the network broadly or replacing the upstream edge unnecessarily, a dedicated and controlled internal security domain was built behind the existing router. This made the final solution both secure and operationally reliable for a home and IoT environment.